

AKILLI SAHA CRM: SIFIR MALİYETLİ PoC & YOL HARİTASI

Ekip İçi Teknik Kılavuz, Tooling, Güvenlik Mimarisi ve Azure Geçiş Stratejisi

Proje: Akıllı Saha CRM (Sıfır Maliyetli PoC & Azure-Ready Mimari)

Belge Türü: Ekip İçi Teknik Yol Haritası ve Görev Dağılımı

Hedef: 0 TL Altyapı ile Canlıya Çıkış -> Azure Enterprise Geçiş

Tarih / Durum: Ekim 2026 / Geliştirme Başlangıcı

1. Mühendislik Stratejisi: 'Azure-Ready & Zero-Cost PoC'

Bu belgenin amacı, gıda distribütörü CRM projesini geliştirici ekibimiz için en net ve pratik adımlara bölmektir. Prototip aşamasında altyapı maliyetini 0 TL seviyesinde tutacağız; ancak kesinlikle geçici veya spagetti kod yazmayacağız. Tam aksine, 'Cloud-Agnostic' ve 'Azure-Ready' prensiplerine sadık kalarak, ürün satıldığı gün tek bir satır iş mantığı (business logic) değiştirmeden Azure Container Apps ve Azure Database for PostgreSQL ortamına taşınabilecek kurumsal bir iskelet kuracağız.

2. Kullanılacak Araçlar, Kütüphaneler ve Servisler (Tech Stack & Tooling)

Tüm ekip üyelerinin standart bir geliştirme ortamında çalışması için belirlenen resmi araç seti:

Katman / Alan	Kullanılacak Teknoloji / Kütüphane	Görev ve Rolü
Core Backend	Java 21, Spring Boot 3.x, Spring Data JPA, Spring Security, jjwt (JWT)	REST API mimarisi, DTO katmanı, iş mantığı ve token bazlı çoklu kullanıcı yetkilendirmesi.
Database (PoC)	PostgreSQL (Supabase Free Tier) + Flyway Migrations	ACID uyumlu ilişkisel veritabanı. Tablo sürümleri Flyway SQL scriptleri ile kod tabanında tutulur.
Frontend (Web SPA)	React (Vite), TypeScript, Tailwind CSS, Lucide Icons, Axios	Hızlı, tip güvenli modern kullanıcı arayüzü; firma, ziyaret ve teklif formları.
Konteyner & Ortam	Docker & Multi-Stage Dockerfile	Uygulamanın hem yerelde, hem Render'da hem de yarın Azure'da birebir aynı çalışmasını sağlayan konteyner yapısı.
Hosting & Keep-Alive	Render (Backend), Vercel (Frontend), UptimeRobot (Ping)	Sıfır maliyetle 7/24 uyanık kalan, SSL sertifikalı çalışan canlı demo

3. Manuel Güvenlik Mimarisi (Security by Design)

Bulut düzeyindeki pahalı güvenlik duvarlarına para ödmeden önce, uygulama katmanında zırh gibi bir koruma sağlayacağız. Kodlarımız şu güvenlik standartlarına sıkı sıkıya bağlı kalacaktır:

- **Parola Kriptografisi:** Kullanıcı parolaları veritabanında asla düz metin (plain text) olarak saklanmayacak, BCryptPasswordEncoder ile tuzlanıp şifrelenecektir.
- **Stateless JWT Doğrulaması:** Her API isteğinde Authorization: Bearer <token> başlığı zorunlu tutulacak. Token süresi (TTL) kısıtlı olacak ve kullanıcı oturumu güvenli şekilde kontrol edilecektir.
- **İstek Bazlı Kimlik Tespiti:** Sistemin çoklu kullanıcı (Multi-User) güvenliği için Spring Security SecurityContextHolder kullanılacaktır. Giriş yapan kullanıcının kimliği ve rolü (ADMIN veya SALES_REP) her istekte doğrulanacaktır.
- **Veri İzolasyonu (Data Isolation):** Bir temsilci başka bir temsilcinin müşterisini veya ziyaretini ASLA çekemez ve güncelleyemez. Spring Data JPA repository katmanında findByAssignedUser... sorguları ve Controller seviyesinde @PreAuthorize kontrolleriyle veri izolasyonu garanti altına alınacaktır.
- **SQL Injection ve XSS Kalkanı:** Tüm SQL sorguları JPA/Hibernate üzerinden parametrik yürütülecek; frontend'den gelen tüm girdiler DTO seviyesinde Jakarta Validation (@NotNull, @NotBlank, @Size) ile doğrulanacaktır.

4. Azure'a Kusursuz Geçiş Mimarisi (Azure-Ready Coding)

Ürünün satışı yapıldığında Azure'a geçişin dakikalar içinde tamamlanması için ekibin uyması gereken 3 altın kural:

- **1. Konfigürasyon İzolasyonu (12-Factor App):** Kodun içine kesinlikle statik veritabanı URL'i, kullanıcı adı veya şifre yazılmayacaktır (Hardcoding Yasaktır). Tüm ayarlar application.yml içinde \${DB_URL}, \${DB_USER}, \${DB_PASSWORD}, \${JWT_SECRET} şeklinde Environment Variable (Çevre Değişkeni) olarak tanımlanacaktır. Azure'a geçerken sadece bu değişkenler güncellenecektir.
- **2. Otomatik Şema Göçü (Flyway):** Veritabanı tabloları manuel olarak elle açılmayacak, src/main/resources/db/migration klasörü altında V1__init_schema.sql şeklinde tutulacaktır. Azure Database for PostgreSQL açıldığında uygulama ayağa kalkarken tüm şemayı otomatik inşa edecektir.
- **3. Standart Konteynerizasyon:** Backend için yazılacak multi-stage Dockerfile hem yerel Docker'da hem Render'da hem de Azure Container Registry (ACR) üzerinde derlenip Azure Container Apps'e doğrudan deploy edilebilir standartta olacaktır.

5. Detaylı Yol Haritası ve Zaman Planı (Timeline & Workload)

Sıfır maliyetli PoC'nin geliştirilmesi, test edilmesi ve canlıya alınması 3 ana faza bölünmüştür. Toplam geliştirme süresi 15 iş günüdür (3 Hafta):

Faz / Aşama	Odak ve Yapılacak	Kullanılacak Toollar	Efor / Süre
-------------	-------------------	----------------------	-------------

İşler			
Faz 1: Veritabanı & Çekirdek Güvenlik	PostgreSQL şeması (Users, Company, Visit, Opportunity), Spring Boot iskeleti, JWT Auth, Password Encoding ve DTO validasyonları.	Java, Spring Boot, Supabase, Flyway, Postman	4 - 5 Gün
Faz 2: İş Mantığı & Web Arayüzü	Firma/Ziyaret/Numune CRUD API'leri, React Vite arayüzü, Tailwind formları, Auth Context ve API entegrasyonu.	React, TypeScript, Tailwind, Axios, Spring Data	5 - 6 Gün
Faz 3: Canlı Dağıtım & Keep-Alive	Dockerfile hazırlığı, Render backend deploy, Vercel frontend yayını, UptimeRobot ping kurulumu ve uçtan uca testler.	Docker, Render, Vercel, UptimeRobot	3 - 4 Gün

Hedef Teslimat: Geliştirme Planı: Toplam 3 hafta (15 İş Günü) sonunda müşteri firmanın önüne 'https://saha-crm.vercel.app' şeklinde canlı, çalışan ve şifreyle girilen bir prototip koyulacaktır.

6. Sonuç ve Görev Başlangıcı

Bu stratejiyle ekibimiz; şirkete veya cebimize 1 TL dahi maliyet çıkarmadan kurumsal kalitede bir sistem inşa edecektir. Ürünü müşteriye gösterip satışı kapattığımızda ise yapacağımız tek işlem, Azure üzerinde sanal sunucu ve veritabanı açıp Docker imajımızı oraya yönlendirmek olacaktır.